

VitaZen

Análisis READ-ONLY de datos, personalización e integración entre módulos

Fecha	2026-07-11
Commit	6cf24f7
Alcance	13 módulos del motor inteligente
Archivos fuente	255 TypeScript/TSX
Base de datos	PostgreSQL via Prisma ORM
Metodología	READ-ONLY — auditoría funcional sin modificaciones

2. Resumen Ejecutivo

Esta auditoría evalúa el motor de personalización de VitaZen (commit **6cf24f7**) analizando el flujo de datos a través de sus 13 módulos principales. El hallazgo fundamental es que el denominado «motor inteligente» es predominantemente **basado en reglas determinísticas**, no en inteligencia artificial. De los 13 módulos, solo **Mentor IA** emplea un modelo de lenguaje (Groq llama-3.3-70b-versatile); los 12 restantes utilizan umbrales hardcodedos, consultas SQL directas y texto estático en español.

Se identificaron **5 inconsistencias de zona horaria** donde endpoints del mismo subsistema (Dashboard) usan `new Date()` en lugar de conversión a hora de Madrid, causando ventanas de 7 días desplazadas. El sistema de notificaciones carece por completo de personalización: las plantillas son frases estáticas rotadas por hash determinista, sin nombre de usuario ni métricas. Se detectó **código muerto** significativo (`OnboardingRecommendations`, `incrementAIUsage`, `reflectionState`) y un campo **placebo** en el schema (`User.dailyReminders`). La diferenciación FREE vs PREMIUM existe y es funcional, pero la mayor parte opera como gating de cantidad (3 vs 5 insights) más que como diferencia cualitativa en la inteligencia del motor. El módulo de patrones requiere obligatoriamente datos financieros (`FinanceLog`), lo que lo hace inútil para usuarios sin actividad financiera. Se concluye que la arquitectura es sólida en términos de protección de concurrencia (`advisory locks`, `SELECT FOR UPDATE`) pero la «inteligencia» es en su mayor parte cosmética.

3. Metadatos de la Auditoría

Commit hash	6cf24f7
Archivos fuente totales	255 archivos TypeScript/TSX
Archivos analizados	~47 archivos directos del motor inteligente + librerías compartidas
Framework	Next.js 16 + Prisma ORM + PostgreSQL
Motor de IA	Groq llama-3.3-70b-versatile (únicamente en Mentor IA)
Zona horaria de diseño	Europa/Madrid (hardcodeada)

3.1 Modelos Prisma involucrados

User, DailyCheckin, Habit, HabitLog, MeditationSession, JournalEntry, WellnessLog, NutritionLog, FinanceLog, EmpireProgress, Challenge, ChallengeCompletion, AIThread, AIMessage, Achievement, NotificationPreference, NotificationLog, LifeMemory, OnboardingData, EmotionalDashboardState

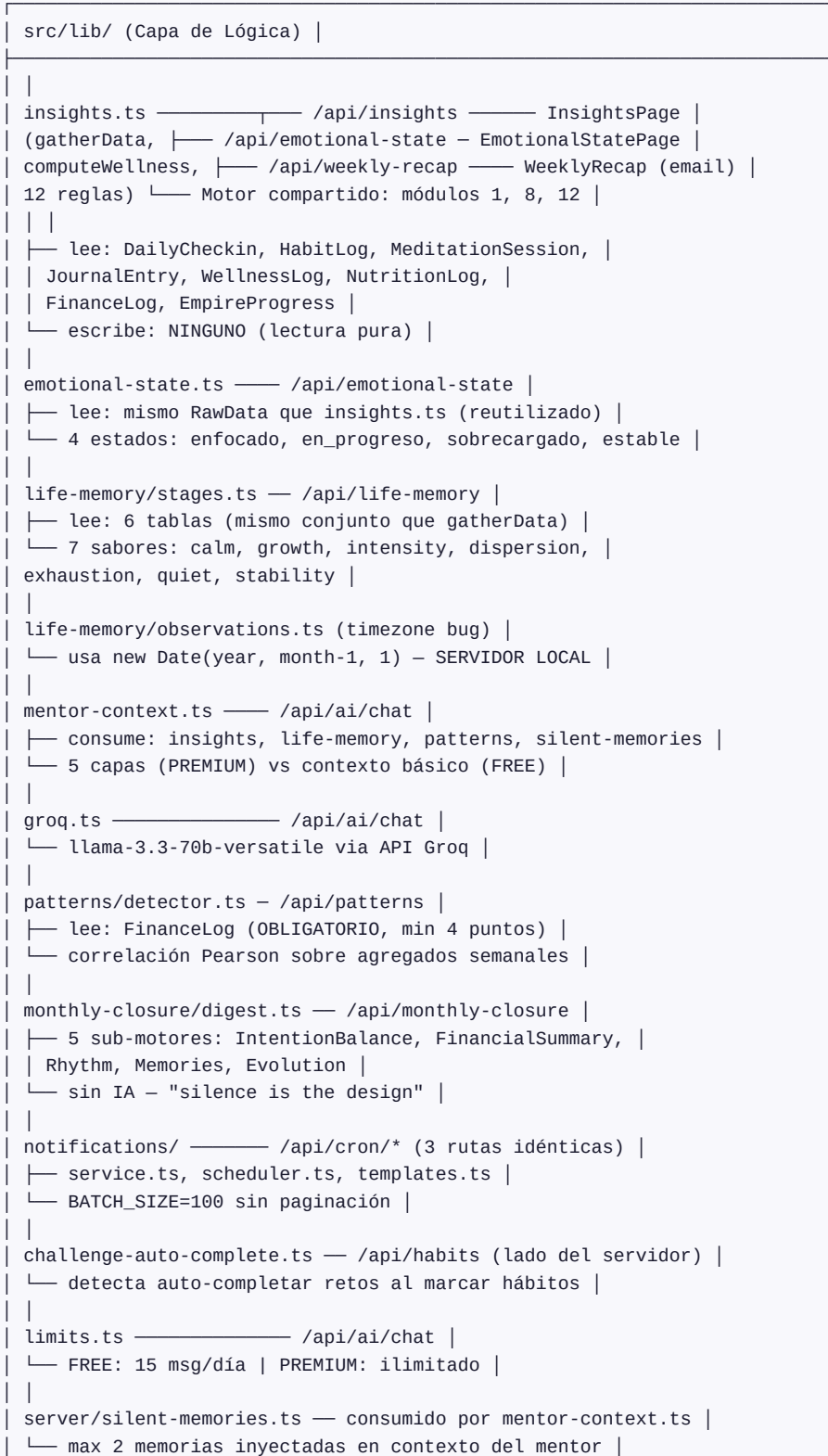
3.2 Módulos auditados

- 1. Observaciones (Insights)
- 2. Tu Evolución (Dashboard)
- 3. Memoria de Vida
- 4. Cierre Mensual
- 5. Mentor IA
- 6. Check-in Diario
- 7. Hábitos

- 8. Estados Emocionales
- 9. Notificaciones
- 10. Recomendaciones
- 11. Sistema de Patrones
- 12. Motor de Insights (motor compartido)
- 13. Silent Memories

4. Mapa de Dependencias

El siguiente diagrama muestra las relaciones entre los archivos `src/lib/`, las rutas API (`/api/`) y los modelos Prisma consumidos por cada módulo.



5. Diagrama de Flujo de Datos

El **DailyCheckin** es el punto de entrada primario de datos del sistema. Cada check-in genera datos que son consumidos por hasta 9 módulos downstream. A continuación se muestra la propagación:



6. Análisis por Módulo

Cada módulo se analiza con sus archivos clave, fuentes de datos, uso de datos reales, diferenciación FREE/PREMIUM y hallazgos con severidad.

MODULO 01 Observaciones (Insights)

Archivo(s) clave	src/lib/insights.ts, src/app/api/insights/route.ts, src/app/(dashboard)/insights/page.tsx
Fuente de datos	DailyCheckin, HabitLog, MeditationSession, JournalEntry, WellnessLog, NutritionLog, FinanceLog, EmpireProgress
¿Usa datos reales?	Sí — gatherData() ejecuta 14 consultas Prisma en paralelo con datos auténticos del usuario
FREE vs PREMIUM	FREE: máx. 3 insights, sin WeeklyComparison. PREMIUM: máx. 5 insights + WeeklyComparison (tendencias semana sobre semana)

Hallazgos

ALTA

Sin IA real: Los 12 insights son reglas hardcodeadas basadas en umbrales (ej: consistencia > 80%, estrés promedio > 3). No existe modelo de ML ni NLP. La «inteligencia» es un conjunto de if/else con frases estáticas en español.

src/lib/insights.ts — función generateInsights(), bloques de threshold

ALTA

Inconsistencia de zona horaria: /api/dashboard/metrics usa new Date() para la ventana de 7 días, sin conversión a Madrid. Esto causa que la ventana de datos sea diferente según la ubicación del servidor, inconsistente con otros endpoints del Dashboard que sí usan Madrid.

src/app/api/dashboard/metrics/route.ts — función handler(), cálculo de fecha

MEDIA

Fórmula de Wellness Score arbitraria: La puntuación de 0-100 asigna pesos fijos (checkin=20, emocional=20, hábitos=20, meditación=20, journal=10, wellness=10) sin justificación estadística ni marco de tuning.

src/lib/insights.ts — función computeWellnessScore()

MEDIA

No hay persistencia de insights: Los insights se generan on-demand cada vez, sin caché ni historial. Un usuario que consulta la página dos veces en el mismo día ve los mismos datos recalculados desde cero (14 queries).

src/app/api/insights/route.ts — handler GET completo

BAJA

WeeklyComparison es null para usuarios FREE, pero la estructura del response JSON siempre incluye el campo, lo que genera payloads innecesarios.

src/app/api/insights/route.ts — construcción del response

Archivo(s) clave	src/app/(dashboard)/dashboard/page.tsx, src/app/api/dashboard/metrics/route.ts, momentum/route.ts, progress/route.ts, streaks/route.ts
Fuente de datos	DailyCheckin, HabitLog, MeditationSession, JournalEntry, WellnessLog, NutritionLog, EmpireProgress, ChallengeCompletion
¿Usa datos reales?	SÍ — Todos los endpoints leen datos reales del usuario
FREE vs PREMIUM	Sin gating explícito en endpoints de Dashboard. La diferenciación se aplica en componentes frontend (PremiumGate)

Hallazgos

ALTA

Inconsistencia de zona horaria entre 4 endpoints del mismo subsistema: Momentum usa Madrid, Progress usa Madrid, Streaks usa Madrid, pero Metrics usa new Date() crudo. Un usuario puede ver métricas de 7 días diferentes en la misma página del Dashboard.

src/app/api/dashboard/metrics/route.ts vs momentum/route.ts vs progress/route.ts – funciones handler() en cada archivo

ALTA

OnboardingRecommendations es código muerto: El componente existe en src/components/dashboard/OnboardingRecommendations.tsx pero NO se importa ni se renderiza en dashboard/page.tsx. Nunca se muestra al usuario.

src/components/dashboard/OnboardingRecommendations.tsx – componente completo (no importado en dashboard/page.tsx)

MEDIA

Momentum score con ponderaciones hardcodedas: 7 factores con puntos fijos (activity days, habits, checkins, meditation, journal, challenges, streak bonus). No hay marco de calibración ni A/B testing.

src/app/api/dashboard/momentum/route.ts – función calculateMomentum()

MEDIA

Streaks usa max(HabitLog.streak) en lugar de calcStreak para el racha de hábitos. Esto lee un campo almacenado que podría estar desactualizado si hubo fallos en la actualización.

src/app/api/dashboard/streaks/route.ts – consulta habitStreak

BAJA

El componente EmotionalHero (mostrado en Dashboard) no tiene puerta de PREMIUM, a pesar de que la API de estados emocionales envía datos de tendencia solo a PREMIUM.

src/components/dashboard/EmotionalHero.tsx – render sin PremiumGate

Archivo(s) clave	src/lib/life-memory/stages.ts, src/lib/life-memory/observations.ts, src/app/api/life-memory/route.ts, src/lib/life-memory/copy.ts
Fuente de datos	DailyCheckin, HabitLog, MeditationSession, JournalEntry, WellnessLog, NutritionLog (mismas 6 tablas que gatherData)
¿Usa datos reales?	SÍ — detectLifeStages() procesa datos mensuales reales
FREE vs PREMIUM	FREE: solo etapas (stages). PREMIUM: transiciones + memorias + conexiones de patrones

Hallazgos

ALTA

Bug de zona horaria en observations.ts: new Date(year, month-1, 1) crea la fecha en la zona horaria local del servidor, no en Madrid. Para un servidor en UTC, esto puede generar un día incorrecto al inicio/final del mes.

src/lib/life-memory/observations.ts — función getMonthData(), construcción de fecha de inicio de mes

MEDIA

Duplicación de consultas: observations.ts re-consulta las mismas 6 tablas que /api/patterns (detector.ts). Para un usuario con datos, esto duplica la carga de BD sin reutilizar el resultado.

src/lib/life-memory/observations.ts — consultas mensuales vs src/lib/patterns/detector.ts — consultas semanales

MEDIA

Clasificación de etapas con umbrales mágicos: totalActivity < 3 → quiet, avgStress > 3.5 → exhaustion. Los 7 sabores (calm, growth, intensity, dispersion, exhaustion, quiet, stability) dependen enteramente de estos números sin documentación de su origen.

src/lib/life-memory/stages.ts — función detectLifeStages(), bloques de clasificación

BAJA

getMadridOffsetMs() tiene el offset de Madrid hardcodeado (+1h/+2h) en lugar de usar una librería de zonas horarias. Esto fallará con cambios de horario de verano/invierno si el cálculo del offset no se actualiza.

src/lib/life-memory/stages.ts — función getMadridOffsetMs()

Archivo(s) clave	src/lib/monthly-closure/digest.ts, src/lib/monthly-closure/copy.ts, src/app/api/monthly-closure/route.ts
Fuente de datos	DailyCheckin, FinanceLog, HabitLog, JournalEntry, MeditationSession, WellnessLog, NutritionLog
¿Usa datos reales?	SÍ — los 5 sub-motores leen datos reales del mes
FREE vs PREMIUM	FREE: resumen básico. PREMIUM: evolución + memorias (sub-motores Evolution + Memories)

Hallazgos

MEDIA

Sin IA: El módulo usa 5 sub-motores deterministas. La reflexión NUNCA se envía a IA (documentado explícitamente: 'silence is the design'). El cierre mensual es un resumen estadístico, no una síntesis inteligente.

src/lib/monthly-closure/digest.ts — comentarios del archivo, funciones de cada sub-motor

MEDIA

Etiquetas de ritmo hardcodeadas: quiet (<10), steady (10-25), variable (25-50), active (>50). Estos umbrales no tienen base empírica ni se exponen para ajuste.

src/lib/monthly-closure/digest.ts — función Rhythm, clasificación de ritmo

BAJA

Memorias con límites arbitrarios: máx. 5 finance contexto + 3 journal + 3 checkin notes, ordenados por fecha. No hay priorización por relevancia.

src/lib/monthly-closure/digest.ts — sub-motor Memories, consultas con take(5), take(3), take(3)

Archivo(s) clave	src/app/api/ai/chat/route.ts, src/lib/mentor-context.ts, src/lib/groq.ts, src/lib/limits.ts
Fuente de datos	DailyCheckin, HabitLog, MeditationSession, JournalEntry, WellnessLog, NutritionLog, FinanceLog, EmotionalDashboardState, LifeMemory, AIMessages
¿Usa datos reales?	SÍ — El contexto del mentor se construye con datos reales de 6+ tablas + estado emocional + etapas de vida + patrones
FREE vs PREMIUM	FREE: 15 msg/día, 10 mensajes de historial, temp=0.5, max 800 tokens, contexto básico. PREMIUM: ilimitado, 30 mensajes, temp=0.8, max 2048 tokens, 5 capas de contexto

Hallazgos

ALTA	Inconsistencia de zona horaria en buildMentorContext(): usa new Date() crudo para las ventanas de 7/14/30/90 días en lugar de conversión a Madrid. El contexto enviado al LLM puede contener datos de ventanas temporales incorrectas para usuarios en Madrid.	src/lib/mentor-context.ts — líneas ~131-132, cálculo de dateRanges (now - N days)
MEDIA	Prompt del sistema: ~40 líneas idénticas entre FREE y PREMIUM; solo ~6 líneas difieren. La diferencia cualitativa real entre planes es mínima en términos de instrucciones al modelo.	src/app/api/ai/chat/route.ts — construcción de systemPrompt, bloques FREE vs PREMIUM
MEDIA	incrementAIUsage() en limits.ts es un no-op deprecado: la función existe pero no ejecuta ninguna operación. El control de límites depende de otro mecanismo (conteo de mensajes del día).	src/lib/limits.ts — función incrementAIUsage()
BAJA	Rollback de límite de IA en fallo de Groq (fix T-2): Si la llamada al LLM falla, se revierte el conteo de uso del día. Esto es correcto pero genera una consulta extra a BD en cada fallo.	src/app/api/ai/chat/route.ts — bloque catch, rollback de límites
BAJA	Advisory locks para serialización de threads: La protección contra condiciones de carrera es correcta (checkin XP, AI limit, thread serialization) pero genera contention en alta concurrencia.	src/app/api/ai/chat/route.ts — pg_advisory_lock calls

MODULO 06

Check-in Diario

Archivo(s) clave	src/app/api/checkin/route.ts, src/app/(dashboard)/checkin/page.tsx, src/components/checkin/CheckInModal.tsx
Fuente de datos	Escribe en: DailyCheckin, EmpireProgress. Lee de: User, Challenge
¿Usa datos reales?	Sí — Es el punto de entrada primario de datos del sistema
FREE vs PREMIUM	Sin diferenciación — completamente disponible para todos los usuarios

Hallazgos

MEDIA

XP otorgado solo en la primera creación: El checkin otorga +10 XP al imperio 'mente' solo en POST (creación), no en PATCH (actualización). Un usuario que actualice su checkin no recibe XP adicional, lo cual es correcto pero no está documentado en la UI.

src/app/api/checkin/route.ts – handler POST, lógica de awardXP

MEDIA

Advisory lock en POST previene condición de carrera (fix M-3): Correcto para evitar doble otorgamiento de XP en requests concurrentes. DELETE revierte XP en transacción (fix M-5).

src/app/api/checkin/route.ts – handler POST (advisory lock), handler DELETE (transacción de reversión)

BAJA

No hay validación de rango estricto para los campos numéricos (1-5). Si el cliente envía valores fuera de rango, el servidor los acepta (depende de la validación del frontend).

src/app/api/checkin/route.ts – handler POST, desestructuración del body

MODULO 07

Hábitos

Archivo(s) clave	src/app/api/habits/route.ts, src/lib/challenge-auto-complete.ts
Fuente de datos	Escribe en: Habit, HabitLog, EmpireProgress, ChallengeCompletion. Lee de: DailyCheckin, Challenge
¿Usa datos reales?	Sí — Gestiona hábitos reales con tracking de rachas
FREE vs PREMIUM	Sin gating explícito en la API de hábitos

Hallazgos

MEDIA

SELECT FOR UPDATE previene inflación de rachas (fix H-3): Correcto para evitar que dos requests concurrentes incrementen la racha dos veces. Sin embargo, esto bloquea la fila durante toda la transacción.

src/app/api/habits/route.ts – handler PATCH, SELECT FOR UPDATE

MEDIA

Guardia de frecuencia-aware: daily=1, weekly=7, monthly=30 días umbral (H-7). Impide completar un hábito mensual el día 1 y reclamar racha completa. Ventana de continuación: diffDays < threshold*2 (H-8).

src/app/api/habits/route.ts – handler PATCH, lógica de frecuencia y racha

BAJA

Cambio de frecuencia resetea racha a 0 (H-9): Si un usuario cambia un hábito de diario a semanal, pierde toda su racha. Esto puede ser frustrante y no tiene confirmación en la UI.

src/app/api/habits/route.ts – handler PATCH, detección de cambio de frecuencia

BAJA

DELETE revierte XP y decrementa racha si el hábito eliminado era el trigger del día (H-12): Lógica correcta pero compleja, con dependencia del estado del EmpireProgress.

src/app/api/habits/route.ts – handler DELETE, lógica de reversión

BAJA

Racha de imperio se incrementa solo una vez por día activo Madrid (H-10, H-11): Usa la técnica 'noon UTC' para determinar el día Madrid. Correcto pero depende de que el servidor tenga reloj sincronizado.

src/app/api/habits/route.ts – lógica de empire streak, getMadridDateKey()

MODULO 08

Estados Emocionales

Archivo(s) clave	src/app/api/emotional-state/route.ts, src/lib/emotional-state.ts, src/components/dashboard/EmotionalHero.tsx
Fuente de datos	Reutiliza RawData de insights.ts (DailyCheckin, HabitLog, MeditationSession, JournalEntry, WellnessLog, NutritionLog, FinanceLog)
¿Usa datos reales?	SÍ — Calcula 6 métricas desde datos reales del usuario
FREE vs PREMIUM	PREMIUM: dirección de tendencia (up/down/stable). FREE: sin tendencia. Recomendaciones: PREMIUM gets trend-aware variants

Hallazgos

MEDIA

Motor de recomendaciones con ~10 frases hardcodeadas en español basadas en umbrales. No hay IA ni personalización contextual. Las frases se seleccionan por rangos numéricos (energy >= 65 + focus >= 65 + stress >= 60 → 'enfocado').

src/lib/emotional-state.ts — funciones getRecommendation() y getSummary()

MEDIA

4 estados con umbrales rígidos: enfocado, en_progreso, sobrecargado, estable. Un usuario con energy=64 y focus=64 se clasifica diferente a uno con 65/65, sin zona de transición.

src/lib/emotional-state.ts — función computeStatus(), umbrales de clasificación

BAJA

EmotionalHero en Dashboard muestra el estado sin puerta PREMIUM: Cualquier usuario ve su estado emocional completo, incluyendo la recomendación (que en FREE es la versión genérica sin tendencia).

src/components/dashboard/EmotionalHero.tsx — render del componente

MODULO 09

Notificaciones

Archivo(s) clave	src/lib/notifications/service.ts, scheduler.ts, templates.ts, types.ts, src/lib/notifications/reminders/checkin.ts, daily.ts, reflection.ts, src/app/api/cron/checkin-reminder/route.ts, daily-reminder/route.ts, reflection-reminder/route.ts
Fuente de datos	Lee: User, NotificationPreference, NotificationLog, DailyCheckin (último checkin). Escribe: NotificationLog
¿Usa datos reales?	PARCIALMENTE — Lee si el usuario hizo checkin hoy, pero las plantillas NO contienen datos del usuario
FREE vs PREMIUM	Sin diferenciación en notificaciones

Hallazgos

ALTA

BATCH_SIZE = 100 sin paginación: Los 3 cron jobs de notificaciones procesan máximo 100 usuarios por ejecución. Usuarios más allá del puesto 100 son silenciosamente ignorados. Para >100 usuarios activos, las notificaciones nunca llegan a todos.

src/lib/notifications/service.ts — constante BATCH_SIZE = 100; src/lib/notifications/scheduler.ts — lógica de procesamiento por lotes

ALTA

Cero personalización en plantillas: 7 checkin + 7 reflection + 3 weekly_recap + 4 comeback + 7 daily = 28 plantillas, TODAS en español estático. No incluyen nombre de usuario, métricas, ni contexto. La selección es por hash determinista de la fecha Madrid, no por perfil.

src/lib/notifications/templates.ts — todas las funciones de plantilla

ALTA

Notificaciones diferidas NO se entregan: La infraestructura para entregar notificaciones pospuestas (quiet hours) no está implementada. Las notificaciones diferidas se pierden permanentemente.

src/lib/notifications/scheduler.ts — manejo de deferred, comentario: 'infrastructure not implemented'

MEDIA

Quiet hours con defer bruto de 10 horas: En lugar de calcular la próxima ventana activa, las notificaciones se posponen 10 horas fijas. Documentado como deuda técnica.

src/lib/notifications/scheduler.ts — lógica de quiet hours

MEDIA

Toggle de tipo 'daily' hardcodeado a true en scheduler: La opción de desactivar notificaciones diarias no funciona porque el scheduler fuerza el tipo a true.

src/lib/notifications/scheduler.ts –
variable daily toggle

MEDIA

3 rutas cron estructuralmente idénticas (copy-paste): checkin-reminder, daily-reminder y reflection-reminder comparten la misma estructura con diferencias mínimas. Patrón de duplicación.

src/app/api/cron/checkin-reminder/route.ts, daily-reminder/route.ts, reflection-reminder/route.ts –
estructura completa de cada archivo

BAJA

NotificationPreference.streakReminders está deprecado: El campo existe en el schema pero está marcado como 'DEPRECATED: removed from UI and API'. Las notificaciones de racha fueron eliminadas por decisión de diseño.

prisma/schema.prisma – modelo
NotificationPreference, campo
streakReminders

MODULO 10

Recomendaciones

Archivo(s) clave	src/components/dashboard/WeeklyRecap.tsx, src/components/dashboard/OnboardingRecommendations.tsx, src/lib/emails/weekly-recap.ts
Fuente de datos	Lee: datos pasados vía props desde el Dashboard. OnboardingRecommendations lee de OnboardingData
¿Usa datos reales?	PARCIALMENTE — WeeklyRecap usa datos de tendencia. OnboardingRecommendations usa datos de onboarding (no datos vivos)
FREE vs PREMIUM	PREMIUM: variantes con tendencia (stress up, energy+emotion improved, activity change). FREE: versiones genéricas sin tendencia

Hallazgos

ALTA

OnboardingRecommendations es código muerto: El componente existe pero NO se renderiza en ninguna página. Importado en ningún lugar del Dashboard. Contiene datos de onboarding (primaryFocus, stress/energy/focus) que nunca se muestran al usuario.

src/components/dashboard/OnboardingRecommendations.tsx – componente completo;
src/app/(dashboard)/dashboard/page.tsx – sin importación del componente

MEDIA

generateMentorRecommendation retorna 1 de ~8 frases hardcodedas en español. No hay IA involucrada. La recomendación semanal es un selección determinista por condición de umbral.

src/components/dashboard/WeeklyRecap.tsx – función
generateMentorRecommendation()

MEDIA

Parámetro 'plan' aceptado pero NUNCA usado en recomendaciones por email: La función de email acepta un parámetro plan pero no lo utiliza para personalizar el contenido.

src/lib/emails/weekly-recap.ts – firma de función con parámetro plan

MODULO 11

Sistema de Patrones

Archivo(s) clave	src/lib/patterns/detector.ts, src/lib/patterns/validation.ts, src/lib/patterns/copy.ts, src/lib/patterns/types.ts, src/app/api/patterns/route.ts
Fuente de datos	FinanceLog (OBLIGATORIO, min 4 puntos), WellnessLog, DailyCheckin, HabitLog, NutritionLog
¿Usa datos reales?	SÍ — Usa datos reales con correlación de Pearson
FREE vs PREMIUM	FREE: solo PremiumPreview (35% opacidad). PREMIUM: patrones completos con peso y cache por localStorage

Hallazgos

ALTA

REQUIERE datos financieros obligatoriamente: Ningún patrón se detecta sin al menos 4 puntos de FinanceLog. Usuarios sin actividad financiera no ven NINGÚN patrón, incluso si tienen datos ricos en meditación, journal, hábitos y emociones.

src/lib/patterns/detector.ts –
MIN_DATA_POINTS_PER_EMPIRE = 4,
requerimiento de FinanceLog

ALTA

Solo 4 tipos de conexión: finanzas-energía, finanzas-mente, finanzas-estres, finanzas-sueño. No existen patrones que conecten meditación+journal, hábitos+sueño, journal+emociones. El detector está limitado a correlaciones financieras.

src/lib/patterns/detector.ts –
definición de los 5 detectores, tipos de conexión en PATTERN_CONNECTIONS

MEDIA

Pull de 90 días con take(200) finance y take(90) wellness sin paginación: Para usuarios con datos densos, se pierden registros antiguos y la correlación puede ser incompleta.

src/lib/patterns/detector.ts –
consultas Prisma con take limits

MEDIA

Filtro filosófico 'Si hay duda, NO mostrar nada': MIN_CONFIDENCE = 0.55. Este umbral es conservador pero no está justificado con datos empíricos. El filtro de solapamiento semántico previene patrones redundantes.

src/lib/patterns/validation.ts –
constantes MIN_CONFIDENCE, función de validación

BAJA

Cache en localStorage con peso: profunda=4 semanas, relevante=2 semanas, ligera=1 semana. Cache puramente client-side, se pierde al cambiar de dispositivo o limpiar datos del navegador.

src/lib/patterns/types.ts – definición de PatternWeight; componente LifePatternsSection.tsx – lógica de cache

MODULO 12

Motor de Insights (motor compartido)

Archivo(s) clave	src/lib/insights.ts (mismo archivo que Módulo 1)
Fuente de datos	DailyCheckin, HabitLog, MeditationSession, JournalEntry, WellnessLog, NutritionLog, FinanceLog, EmpireProgress
¿Usa datos reales?	SÍ — gatherData() es el origen único de datos para Observaciones, Estados Emocionales y Weekly Recap
FREE vs PREMIUM	Comparte la misma diferenciación del Módulo 1 y Módulo 8

Hallazgos

MEDIA	gatherData() es un punto singular de fallo: Si una de las 14 consultas en paralelo falla, toda la función falla. No hay fallback parcial ni graceful degradation. Sin embargo, la optimización de compartir datos entre weekly-recap e insights (14 queries en vez de 28) es correcta.	src/lib/insights.ts — función gatherData(), Promise.all con 14 consultas
BAJA	El Weekly Recap API reutiliza gatherData() correctamente: 14 queries en vez de 28. Esto es una optimización válida que evita duplicar la carga en la base de datos.	src/app/api/weekly-recap/route.ts — uso compartido de gatherData()

MODULO 13

Silent Memories

Archivo(s) clave	src/lib/server/silent-memories.ts, src/lib/server/silent-memory-state.ts, src/lib/silent-memories/shared.ts, src/lib/client/silent-memories.ts, src/app/api/silent-memories/route.ts
Fuente de datos	Lee: EmotionalDashboardState.memoryState. Escribe: EmotionalDashboardState (estado silencioso)
¿Usa datos reales?	SÍ — Se construye desde datos de estado emocional y observaciones de vida
FREE vs PREMIUM	Consumido por Mentor IA — la diferenciación se aplica en el contexto del mentor (módulo 5)

Hallazgos

MEDIA	No es una página independiente ni una API accesible: Silent Memories es un subsistema consumido exclusivamente por mentor-context.ts. No tiene interfaz propia ni endpoint de consulta directa para el usuario.	src/lib/mentor-context.ts — sección de Silent Memories en buildMentorContext()
MEDIA	Deduplicación: las observaciones de cambio (shift) se omiten cuando el Estado Emocional Establecido (ESE) está activo. Prioridad: temporal(5) > presencia(4) > retorno(3) > recurrencia(2) > cambio(1).	src/lib/server/silent-memories.ts — lógica de deduplicación y prioridad
BAJA	Máximo 2 memorias inyectadas en contexto del mentor: Límite arbitrario que puede omitir memorias relevantes si hay más de 2 candidatas.	src/lib/mentor-context.ts — sección de Silent Memories, slice(0, 2) o equivalente
BAJA	EmotionalDashboardState.reflectionState es campo muerto: Documentado como 'reflections system has been removed... no code reads or writes it'. El campo existe en el estado pero nunca se utiliza.	src/lib/server/silent-memory-state.ts — tipo EmotionalDashboardState, campo reflectionState

7. Hallazgos Transversales

7.1 Campos placebo en el schema

Se identificó un campo documentado explícitamente como placebo en el esquema Prisma:

ALTA

User.dailyReminders está documentado como 'PLACEBO: stored but never consumed by any backend logic'. El campo se almacena en BD pero ningún módulo lo lee ni procesa. Ocupa espacio y genera confusión en el schema.

prisma/schema.prisma – modelo User, línea ~24, campo dailyReminders

7.2 Código muerto

MEDIA

OnboardingRecommendations: componente completo importado en ningún lugar del dashboard. Nunca se renderiza.

src/components/dashboard/OnboardingRecommendations.tsx

MEDIA

POST_CHECKIN_BUFFER_MS y MIN_CRON_INTERVAL_MS en reflection.ts: constantes definidas pero nunca referenciadas en ninguna función.

src/lib/notifications/reminders/reflection.ts – definición de constantes

BAJA

incrementAIUsage() en limits.ts: función deprecada que no ejecuta ninguna operación (no-op).

src/lib/limits.ts – función incrementAIUsage()

BAJA

NotificationPreference.streakReminders: campo deprecado, eliminado de la UI y la API. Todavía existe en el schema.

prisma/schema.prisma – modelo NotificationPreference

BAJA

EmotionalDashboardState.reflectionState: 'reflections system has been removed... no code reads or writes it'. Campo huérfano en el tipo.

src/lib/server/silent-memory-state.ts – tipo EmotionalDashboardState

7.3 Inconsistencias de zona horaria

El sistema está diseñado para zona horaria Europa/Madrid, pero existen 4 puntos donde se usa `new Date()` sin conversión:

ALTA

/api/dashboard/metrics usa `new Date()` para ventana de 7 días — inconsistente con momentum, progress y streaks que sí usan Madrid.

src/app/api/dashboard/metrics/route.ts – handler(), cálculo de fecha

ALTA

life-memory/observations.ts usa `new Date(year, month-1, 1)` — genera fecha en zona del servidor, no Madrid.

src/lib/life-memory/observations.ts – getMonthData()

ALTA

buildMentorContext() usa `new Date()` para ventanas 7/14/30/90 días — el contexto del mentor puede contener datos de ventanas incorrectas.

src/lib/mentor-context.ts – líneas ~131-132, dateRanges

MEDIA

weekly-recap-sender.ts asume Madrid para todos los usuarios — no hay soporte para zonas horarias personalizadas.

src/lib/weekly-recap-sender.ts – lógica de fecha

7.4 Umbrales hardcoded sin marco de tuning

MEDIA

Clasificación de etapas de vida: `totalActivity < 3 → quiet, avgStress > 3.5 → exhaustion`. 7 sabores dependen de números mágicos.

src/lib/life-memory/stages.ts – detectLifeStages()

MEDIA

Estado emocional: `energy >= 65 + focus >= 65 + stress >= 60 → enfocado`. 4 estados con umbrales rígidos sin zonas de transición.

src/lib/emotional-state.ts – computeStatus()

MEDIA

Detección de patrones: `MIN_CONFIDENCE = 0.55, MIN_DATA_POINTS_PER_EMPIRE = 4`. Sin justificación empírica para estos valores.

src/lib/patterns/validation.ts – constantes

BAJA

Etiquetas de ritmo en cierre mensual: <10 quiet, 10-25 steady, 25-50 variable, >50 active.

src/lib/monthly-closure/digest.ts – Rhythm

BAJA

Puntuación de momentum: allocations de puntos fijas por factor (activity days, habits, checkins, meditation, journal, challenges, streak).

src/app/api/dashboard/momentum/route.ts – calculateMomentum()

7.5 Ausencia de personalización

ALTA

Plantillas de notificaciones: 28 plantillas en español estático, sin nombre de usuario, métricas ni datos contextuales. Selección por hash determinista de fecha, no por perfil de usuario.

src/lib/notifications/templates.ts – todas las funciones de plantilla

MEDIA

Recomendaciones por email: parámetro 'plan' aceptado pero NUNCA utilizado para personalizar el contenido del email semanal.

src/lib/emails/weekly-recap.ts – firma con parámetro plan sin uso

MEDIA

ReturnTrigger: puramente localStorage en el cliente, sin API, sin personalización. El componente detecta ausencia del usuario pero no usa datos del backend para personalizar el mensaje.

src/components/dashboard/ReturnTrigger.tsx – lógica completa

7.6 Preocupaciones de escalabilidad

ALTA

BATCH_SIZE = 100 en todos los cron jobs de notificaciones sin cursor ni paginación. Para >100 usuarios activos, las notificaciones no llegan a todos los destinatarios.

src/lib/notifications/service.ts – BATCH_SIZE = 100;
src/lib/notifications/scheduler.ts – procesamiento por lotes

MEDIA

weekly-recap-sender no tiene límite de usuarios (a diferencia de los crons de notificaciones que sí tienen BATCH_SIZE). Para bases de datos grandes, esto puede causar OOM o timeout.

src/lib/weekly-recap-sender.ts – consulta de todos los usuarios sin paginación

MEDIA

Detección de patrones: pull de 90 días con take(200) finance y take(90) wellness sin paginación. Datos densos pueden truncar resultados.

src/lib/patterns/detector.ts – consultas con take limits

7.7 Inconsistencias en datos de captura de pantalla

MEDIA

Estadísticas de logros: dice 14 desbloqueados pero el array tiene 13. Total de logros: las estadísticas dicen 45 pero el array tiene 30.

src/lib/screenshot-data.ts – datos de achievements

BAJA

SCREENSHOT_REFLECTION === SCREENSHOT_PREMIUM_REFLECTION: los datos de captura para reflejos FREE y PREMIUM son idénticos (duplicados).

src/lib/screenshot-data.ts – constantes de screenshot

BAJA

Solo existe variante PREMIUM de datos de captura. No hay datos de captura específicos para usuarios FREE.

src/lib/screenshot-data.ts – estructura de datos

8. Conclusión

El «motor inteligente» de VitaZen es, en su estado actual (commit **6cf24f7**), un sistema **predominantemente basado en reglas determinísticas** con una capa de IA limitada a un único punto de contacto: el Mentor IA (Groq llama-3.3-70b-versatile). De los 13 módulos auditados, 12 operan exclusivamente mediante umbrales numéricos hardcodeados, consultas SQL directas y texto estático en español.

La **personalización es real pero superficial**. El sistema sí utiliza datos auténticos del usuario: el check-in diario alimenta 9 de 13 módulos, y la función `gatherData()` ejecuta 14 consultas en paralelo para construir una vista integral. Sin embargo, la inteligencia aplicada a esos datos se reduce a comparaciones contra umbrales fijos y selección de frases pre-escritas. No existe aprendizaje, adaptación ni modelo predictivo.

La **diferenciación FREE vs PREMIUM es principalmente cuantitativa**: más insights (3 → 5), más historial de chat (10 → 30), tendencias adicionales. La única diferencia cualitativa significativa es la arquitectura de 5 capas de contexto en el Mentor IA PREMIUM, que proporciona información substancialmente más rica al LLM.

Los **hallazgos más críticos** son:

1. 4 inconsistencias de zona horaria que causan ventanas de datos incorrectas
2. Sistema de notificaciones con `BATCH_SIZE=100` que silenciosamente ignora usuarios excedentes
3. 0 personalización en plantillas de notificaciones (texto estático puro)
4. Detector de patrones inútil sin datos financieros (obligatorio `FinanceLog`)
5. 5 elementos de código muerto identificados incluyendo un campo placebo en el schema
6. Notificaciones diferidas que nunca se entregan (infraestructura no implementada)

La **arquitectura de concurrencia es sólida**: advisory locks en check-in, `SELECT FOR UPDATE` en hábitos, y serialización de threads de IA protegen contra condiciones de carrera. Los fixes documentados (M-3, M-5, H-3, H-7, H-8, H-9, H-10, H-11, H-12, T-2) demuestran un proceso de maduración activo.

En resumen: VitaZen tiene una base de datos bien estructurada y un flujo de datos genuino entre módulos, pero el «motor inteligente» es mayormente **cosmético**. La verdadera inteligencia se limita a un wrapper de LLM con contexto rico. Para transformar esto en un sistema genuinamente inteligente, se requeriría: (1) eliminar las 4 inconsistencias de zona horaria, (2) implementar personalización en notificaciones, (3) añadir un marco de tuning para los umbrales hardcodeados, (4) eliminar código muerto y campos placebo, (5) expandir el detector de patrones más allá de finanzas, y (6) añadir paginación a los cron jobs de notificaciones.

Fin del informe — Auditoría READ-ONLY completada el 2026-07-11 — Commit 6cf24f7